Forward Propagation and Backpropagation

Linear Algebra and Calculus for a NumPy Neural Network

This document explains a fully connected network with arbitrary hidden-layer sizes and one or more outputs. The equations match the implementation in which a weight matrix has one row per incoming value, an extra final row for biases, and one column per destination neuron. The example architecture is $[2, 4, 4, 1]$.

1 Notation and matrix orientation

Number the input layer 0 and the trainable layers $1, \dots, L$, with layer L the output. Let n_ℓ be the number of neurons in layer ℓ. For one training example, use column vectors:

$$\boldsymbol{a}^{(0)} = \boldsymbol{x} \in \mathbb{R}^{n_0}, \qquad \boldsymbol{a}^{(\ell)} \in \mathbb{R}^{n_\ell}, \qquad \boldsymbol{s}^{(\ell)} \in \mathbb{R}^{n_\ell}.$$

Here $\boldsymbol{s}$ is the weighted sum before activation and $\boldsymbol{a}$ is the activated output. The target is $\boldsymbol{t} \in \mathbb{R}^{n_L}$. Write

$$W^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}, \qquad \boldsymbol{b}^{(\ell)} \in \mathbb{R}^{n_\ell}, \qquad \widetilde{W}^{(\ell)} = \begin{bmatrix} W^{(\ell)} \\ (\boldsymbol{b}^{(\ell)})^T \end{bmatrix} \in \mathbb{R}^{(n_{\ell-1}+1) \times n_\ell}. \tag{1}$$

The stored NumPy matrix is $\widetilde{W}^{(\ell)}$. Its element $w_{ij}^{(\ell)}$ connects source neuron i to destination neuron j. Its final row contains one independent bias weight for each destination neuron.

Mathematical quantity	Python expression	Meaning
$\widetilde{W}^{(\ell)}$	`weights[ell - 1]`	weights including biases
$W^{(\ell)}$	`weights[ell - 1][:-1, :]`	ordinary connections
$(\boldsymbol{b}^{(\ell)})^T$	`weights[ell - 1][-1, :]`	bias weights
$\boldsymbol{a}^{(0)}$	`input`	input values
$\boldsymbol{a}^{(\ell)}$, $\ell < L$	`nodes[ell - 1]`	hidden-layer outputs
$\boldsymbol{a}^{(L)}$	`output`	current prediction
$\boldsymbol{s}^{(\ell)}$	`weightedSums[ell - 1]`	pre-activation sums

Python names in this table omit the `self.` prefix. One-dimensional NumPy arrays have shape `(n,)`; they do not have a row/column orientation, and transposing them does not change their shape. The equations use explicit column vectors to make dimensions clear.

2 Forward propagation

For destination neuron j in layer ℓ,

$$s_j^{(\ell)} = \sum_{i=1}^{n_{\ell-1}} w_{ij}^{(\ell)} a_i^{(\ell-1)} + b_j^{(\ell)}, \qquad a_j^{(\ell)} = f\big(s_j^{(\ell)}\big). \tag{2}$$

In matrix form, define the augmented input

$$\overline{\boldsymbol{a}}^{(\ell-1)} = \begin{bmatrix} \boldsymbol{a}^{(\ell-1)} \\ 1 \end{bmatrix}. \tag{3}$$

Then the complete layer calculation is

$$\boxed{\boldsymbol{s}^{(\ell)} = (\widetilde{W}^{(\ell)})^T \overline{\boldsymbol{a}}^{(\ell-1)}, \qquad \boldsymbol{a}^{(\ell)} = f(\boldsymbol{s}^{(\ell)})}. \tag{4}$$

The activation acts element by element. The constant 1 multiplies the bias row, so the matrix multiplication includes every neuron's bias. With one-dimensional NumPy arrays, both following expressions calculate the same weighted-sum vector:

```
previous_with_bias = np.append(previous_output, 1.0)
weighted_sum = previous_with_bias @ weights
# Equivalent for a one-dimensional previous_with_bias:
weighted_sum = weights.T @ previous_with_bias
activated = self.activation.function(weighted_sum)
```

Apply this calculation at every hidden layer and the output layer. Save the weighted sums and hidden outputs for the following backward pass; also assign the current prediction to `self.output`.

2.1 Activation functions and their derivatives

For binary targets such as XOR's 0 and 1, the binary sigmoid is

$$f(s) = \sigma(s) = \frac{1}{1+e^{-s}}, \qquad f'(s) = \frac{e^{-s}}{(1+e^{-s})^2} = \sigma(s)(1-\sigma(s)). \tag{5}$$

Thus, when the activated output is $a = \sigma(s)$, its derivative is $a(1-a)$. The bipolar sigmoid is

$$g(s) = 2\sigma(s) - 1, \qquad g'(s) = 2\sigma(s)(1-\sigma(s)) = \frac{1}{2}(1-g(s)^2). \tag{6}$$

Use targets such as -1 and 1 when training bipolar XOR. The existing `activation.derivative(s)` method expects the pre-activation sum s; a derivative computed as $a(1-a)$ instead expects the activated output a. These arguments must not be interchanged.

2.2 Why hidden activations matter for XOR

Composing affine maps without nonlinear hidden activations produces another affine map. For example,

$$A_2(A_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2 = (A_2A_1)\boldsymbol{x} + (A_2\boldsymbol{b}_1 + \boldsymbol{b}_2). \tag{7}$$

An affine decision boundary cannot separate XOR's two positive corners from its two negative corners. Nonlinear hidden activations allow the network to construct features that separate those cases.

3 Squared error and the output-layer chain rule

For one example, the loss is

$$E = \frac{1}{2}\|\boldsymbol{a}^{(L)} - \boldsymbol{t}\|_2^2 = \frac{1}{2}\sum_{j=1}^{n_L}(a_j^{(L)} - t_j)^2. \tag{8}$$

The factor $1/2$ cancels the factor 2 from differentiating the square:

$$\frac{\partial E}{\partial a_j^{(L)}} = a_j^{(L)} - t_j. \tag{9}$$

Because $a_j^{(L)} = f(s_j^{(L)})$, the chain rule gives

$$\frac{\partial E}{\partial s_j^{(L)}} = \frac{\partial E}{\partial a_j^{(L)}}\frac{\partial a_j^{(L)}}{\partial s_j^{(L)}} = (a_j^{(L)} - t_j)f'(s_j^{(L)}). \tag{10}$$

The lecture notes use the opposite-sign quantity as their error signal:

$$\boldsymbol{\delta}^{(\ell)} = -\nabla_{\boldsymbol{s}^{(\ell)}} E. \tag{11}$$

Consequently the output signal is

$$\boxed{\boldsymbol{\delta}^{(L)} = (\boldsymbol{t} - \boldsymbol{a}^{(L)}) \odot f'(\boldsymbol{s}^{(L)})}. \tag{12}$$

Here $\odot$ means elementwise multiplication. For binary sigmoid, replace $f'(\boldsymbol{s}^{(L)})$ by $\boldsymbol{a}^{(L)} \odot (\boldsymbol{1} - \boldsymbol{a}^{(L)})$.

4 Propagating error through hidden layers

A hidden neuron has no directly supplied target. Its effect on the loss is transmitted through every neuron it connects to in the next layer. Since

$$s_h^{(\ell+1)} = \sum_i w_{ih}^{(\ell+1)} a_i^{(\ell)} + b_h^{(\ell+1)}, \qquad \frac{\partial s_h^{(\ell+1)}}{\partial a_j^{(\ell)}} = w_{jh}^{(\ell+1)}, \tag{13}$$

the multivariable chain rule gives

$$\frac{\partial E}{\partial s_j^{(\ell)}} = \sum_{h=1}^{n_{\ell+1}} \frac{\partial E}{\partial s_h^{(\ell+1)}}\frac{\partial s_h^{(\ell+1)}}{\partial a_j^{(\ell)}}\frac{\partial a_j^{(\ell)}}{\partial s_j^{(\ell)}} \tag{14}$$

$$= f'(s_j^{(\ell)}) \sum_h w_{jh}^{(\ell+1)} \frac{\partial E}{\partial s_h^{(\ell+1)}}. \tag{15}$$

Using the sign convention in Equation (11), this becomes

$$\boxed{\boldsymbol{\delta}^{(\ell)} = (W^{(\ell+1)}\boldsymbol{\delta}^{(\ell+1)}) \odot f'(\boldsymbol{s}^{(\ell)})}. \tag{16}$$

Calculate these vectors from $\ell = L-1$ down to 1. Equivalently, with $D^{(\ell)} = \mathrm{diag}(f'(\boldsymbol{s}^{(\ell)}))$,

$$\boldsymbol{\delta}^{(\ell)} = D^{(\ell)} W^{(\ell+1)} \boldsymbol{\delta}^{(\ell+1)}. \tag{17}$$

This shows backpropagation as repeated multiplication by transposed forward-map Jacobians.

4.1 Why the bias row is excluded here

Equation (16) uses the ordinary connection matrix W, not the augmented matrix $\widetilde{W}$. A bias is an independent parameter fed by a constant 1, so it has no preceding neuron to receive an error signal. Moreover, $\partial b_h/\partial a_j = 0$. In Python:

```
weighted_error = self.weights[layer + 1][:-1, :] @ deltas[layer + 1]
deltas[layer] = (
    weighted_error * self.activation.derivative(self.weightedSums[layer])
)
```

No expansion with `ones_like` and no transpose of the sliced matrix is needed. The next layer's delta is already a vector of the required length.

5 Weight gradients and the outer product

For one connection into layer ℓ,

$$\frac{\partial s_j^{(\ell)}}{\partial w_{ij}^{(\ell)}} = a_i^{(\ell-1)}. \tag{18}$$

Applying the chain rule again,

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \frac{\partial E}{\partial s_j^{(\ell)}} a_i^{(\ell-1)} = -a_i^{(\ell-1)} \delta_j^{(\ell)}. \tag{19}$$

For a bias, $\partial s_j/\partial b_j = 1$, so

$$\frac{\partial E}{\partial b_j^{(\ell)}} = -\delta_j^{(\ell)}. \tag{20}$$

Combining both in the stored matrix gives

$$\boxed{\nabla_{\widetilde{W}^{(\ell)}} E = -\overline{\boldsymbol{a}}^{(\ell-1)} (\boldsymbol{\delta}^{(\ell)})^T}. \tag{21}$$

Define the change direction $C^{(\ell)} = -\nabla_{\widetilde{W}^{(\ell)}} E$. It is an outer product: every source value multiplies every destination delta. For the first layer of $[2, 4, 4, 1]$,

$$C^{(1)} = \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix} \begin{bmatrix} \delta_1 & \delta_2 & \delta_3 & \delta_4 \end{bmatrix} = \begin{bmatrix} x_1\delta_1 & x_1\delta_2 & x_1\delta_3 & x_1\delta_4 \\ x_2\delta_1 & x_2\delta_2 & x_2\delta_3 & x_2\delta_4 \\ \delta_1 & \delta_2 & \delta_3 & \delta_4 \end{bmatrix}. \tag{22}$$

The last row updates the biases. In Python:

```
change = np.outer(previous_with_bias, deltas[layer])
```

Gradient descent with learning rate $\eta > 0$ is therefore

$$\widetilde{W}^{(\ell)} \leftarrow \widetilde{W}^{(\ell)} - \eta \nabla_{\widetilde{W}^{(\ell)}} E = \widetilde{W}^{(\ell)} + \eta C^{(\ell)}. \tag{23}$$

The addition agrees with the notes' (target − output) delta convention. If deltas were instead defined as $+\nabla_{\boldsymbol{s}} E$, the update would subtract the corresponding outer product.

5.1 Shape check for the example network

Layer	$\widetilde{W}$	augmented input	delta	change C
Hidden 1	3×4	3×1	4×1	3×4
Hidden 2	5×4	5×1	4×1	5×4
Output	5×1	5×1	1×1	5×1

The output delta remains a one-element NumPy array. It needs no special broadcasting rule. Compute all deltas before changing any weight matrix, so they refer to the same forward pass and parameter values.

6 Online learning, batch learning, and momentum

Let the dataset contain N examples. An epoch visits all N examples; for XOR, $N = 4$.

6.1 Online learning

For each example p, run forward propagation, calculate $C_p^{(\ell)}$, and immediately apply Equation (23). Later examples use the updated weights. XOR therefore has four updates per epoch.

6.2 Batch learning

Keep all weights fixed while calculating the N examples' changes. For the mean example loss

$$J = \frac{1}{N} \sum_{p=1}^{N} E_p, \qquad \overline{C}^{(\ell)} = \frac{1}{N} \sum_{p=1}^{N} C_p^{(\ell)}, \tag{24}$$

the update is

$$\widetilde{W}^{(\ell)} \leftarrow \widetilde{W}^{(\ell)} + \eta \overline{C}^{(\ell)}. \tag{25}$$

This is one update per epoch. If optimizing the total loss $\sum_p E_p$ instead, use the sum of the changes; this makes the update N times as large at the same learning rate. Averaging is the convention used here.

6.3 Momentum and learning-rate placement

With momentum coefficient μ and update counter k, maintain a previous update matrix $U^{(\ell)}$:

$$U_k^{(\ell)} = \eta C_k^{(\ell)} + \mu U_{k-1}^{(\ell)}, \qquad \widetilde{W}_k^{(\ell)} = \widetilde{W}_{k-1}^{(\ell)} + U_k^{(\ell)}. \tag{26}$$

In batch mode, C_k means the averaged batch change. Initialize $U_0 = 0$. Momentum is applied at each actual update, not at each backward calculation when a batch is accumulating. Choose one convention: return C from `backProp()` and multiply by η when applying it, or return ηC and add it directly. Do not multiply by the learning rate in both places. In this document, `backProp()` returns C.

7 A backward pass matching the equations

The following assumes the forward pass saves `self.output`, `self.nodes`, and `self.weightedSums` for the current example. It returns change directions without mutating weights.

```
def backProp(self, target):
    target = np.asarray(target, dtype=float).reshape(-1)
    output = np.asarray(self.output, dtype=float).reshape(-1)
    if target.shape != output.shape:
        raise ValueError("Target size must match output size")

    deltas = [None] * len(self.weights)
    deltas[-1] = (
        (target - output)
        * self.activation.derivative(self.weightedSums[-1])
    )

    for layer in range(len(self.weights) - 2, -1, -1):
        weighted_error = (
            self.weights[layer + 1][:-1, :] @ deltas[layer + 1]
        )
        deltas[layer] = (
            weighted_error
            * self.activation.derivative(self.weightedSums[layer])
        )

    changes = []
    for layer in range(len(self.weights)):
        previous = self.input if layer == 0 else self.nodes[layer - 1]
        previous_with_bias = np.append(previous, 1.0)
        changes.append(np.outer(previous_with_bias, deltas[layer]))
    return changes
```

8 Measuring error at the end of an epoch

After applying that epoch's updates, evaluate every example again using the final weights. Plot

$$E_{\text{epoch}} = \frac{1}{2} \sum_{p=1}^{N} \sum_{j=1}^{n_L} \left(\widehat{a}_{p,j}^{(L)} - t_{p,j} \right)^2. \tag{27}$$

Prediction and target arrays should both have shape (N, n_L). The scalar computation is

```
error = 0.5 * np.sum((predictions - targets) ** 2)
```

This reported total equals NJ, where J is the mean loss used by an averaged batch update. Their scaling differs, but both evaluate the same prediction errors. Evaluate after updates so every plotted point uses one consistent network, rather than mixing online predictions made with different intermediate weights.

9 Implementation checks

- Assign each example to `self.input`; `self.inputs` is a different attribute.
- Use the current prediction in `self.output` for its backward pass. Dataset prediction history may contain a different example's output.
- Remove bias rows only while propagating hidden errors; include them in forward propagation and weight updates.
- Use random initial weights. Identically initialized neurons receive identical updates and cannot develop distinct features.
- Two sigmoid hidden layers can learn slowly with small initial weights: hidden derivatives and weights multiply repeatedly in Equation (16).
- The gradient describes the local change in loss. A finite step is not guaranteed to reduce loss if its learning rate is too large; online epoch losses need not decrease monotonically.